

Open EM Side-Channel Dataset for ML-KEM (Kyber) Implementations

Alain Alyosha Magazin¹  and Karim M. Abdellatif¹ 

Ledger, Paris, France

{alain.magazin,karim.abdellatif}@ledger.fr

Abstract. We present an open dataset of electromagnetic (EM) traces captured during the decapsulation operation of ML-KEM (Kyber), the key encapsulation mechanism standardised by NIST in FIPS 203. Each trace is windowed on a single pair-pointwise polynomial multiplication, in which the decapsulation key is one of the operands, making it a recurring target of published side-channel key-recovery attacks. The dataset covers three widely used implementations: the CRYSTALS reference implementation, the Cortex-M4 optimised `pqm4` implementation, and the first-order masked `mkm4` implementation, with 200k traces per implementation. For the masked implementation we release the traces of both shares, enabling first-order leakage assessment and share-wise analysis rather than attacks on unprotected code alone. All measurements were taken on an STM32F407 Cortex-M4 microcontroller using a near-field EM probe. Compared to previously published datasets targeting the same operation, which provide power measurements of the unprotected reference implementation only, this dataset contributes an EM modality and covers optimised and masked code. The traces and the associated sensitive variables are distributed as chunked NumPy arrays, so that researchers without access to measurement equipment can reproduce and extend side-channel analyses of ML-KEM.

Keywords: ML-KEM · Kyber · post-quantum cryptography · electromagnetic side-channel analysis · open dataset · masking · `pqm4`.

1 Introduction

For the past few years, the push for the transition to post-quantum cryptography (PQC) has been on the rise. ML-KEM, also known with its pre-standard name Kyber, is one of the NIST-standardised PQC algorithms for key encapsulation[13,2]. It is therefore important to assess the security of the various implementations of this algorithm, notably in regards to side-channel attacks. Side-channel analysis often requires the use of a dedicated and often expensive hardware and software setup. A public dataset lowers this barrier and broadens security assessment of these implementations. The lack of open datasets for the implementation security testing of PQC has also been identified as a gap by the community[1].

Contributions. The main contribution of this work is the creation of a public dataset containing electromagnetic (EM) traces of the decapsulation operation for three different and widely used implementations of ML-KEM. We also provide a detailed description on how to use this dataset for reproducible research on the side-channel security of ML-KEM. Unlike previously published datasets targeting the same operation, which are limited to power measurements of an unprotected reference implementation, this dataset provides an EM modality and additionally covers an optimised implementation and a first-order masked one, for which the traces of both shares are released. These traces are not published in isolation: each of the three sets underpins a practical key-recovery attack that we have carried out and reported separately[12,10,11], so the dataset is known to carry exploitable leakage (Section 4.3).

Intended use cases. The dataset is meant to support, among others: profiled and unprofiled key-recovery attacks on the unprotected implementations; comparison of the exploitability of plain C against hand-written assembly for the same operation; first-order leakage assessment and share-wise analysis of a masked implementation; evaluation of multivariate and higher-order attacks; and benchmarking of deep-learning-based side-channel analysis on a standardised PQC target.

2 Background

2.1 ML-KEM (Kyber)

A lot of research[9,5,7] has been done on the side-channel security of ML-KEM, and specifically on the polynomial multiplication during decapsulation. Indeed, during the decapsulation operation, one of the polynomials involved in the multiplication is the decapsulation key, which led to this operation being a common attack vector. This is why we decided to focus this dataset on the decapsulation operation and more precisely around this multiplication of polynomials. Here is the code for the polynomial multiplication in the ML-KEM algorithm, where we see polynomials *a* and *b* which are the ones in the metadata of the dataset:

```
void poly_basemul(poly *r, const poly *a, const poly *b) {
    for(unsigned int i=0;i<KYBER_N/4;i++) {
        basemul(&r->coeffs[4*i], &a->coeffs[4*i], &b->
            coeffs[4*i], zetas[64+i]);
        basemul(&r->coeffs[4*i+2], &a->coeffs[4*i+2], &b
            ->coeffs[4*i+2], -zetas[64+i]);
    }
}
```

Listing 1.1: Polynomial multiplication in the decapsulation operation

2.2 Target Implementations

- **Reference implementation.** The reference implementation is the one provided and maintained by the CRYSTALS team[3]. It is a plain C implementation of the ML-KEM algorithm, without any optimizations nor countermeasures. We used the 512 security level of the algorithm.
- **pqm4.** The `pqm4`[8] implementation is an optimized implementation of the ML-KEM algorithm for the Cortex-M4 architecture. It is written in C and M4-specific assembly. In particular, the polynomial multiplication is implemented in hand-written assembly code. We used the 512 security level of the algorithm.
- **mkm4 (masked).** The `mkm4`[6] implementation is a masked implementation of the ML-KEM algorithm for the Cortex-M4 architecture. It is also written in C and M4-specific assembly. It implements first-order masking of the decapsulation key. This implementation only supports the 768 security level.

The reference and `pqm4` implementations are captured at the 512 security level, whereas `mkm4` is captured at 768. This is not a design choice: `mkm4` is only available for the 768 parameter set, so no matching security level exists for the masked implementation.

2.3 Related Datasets

Open datasets for the side-channel analysis of PQC implementations have only recently started to appear. The closest to this work is the dataset of Rezaeezade et al.[16], which also targets the pair-pointwise multiplication during Kyber decapsulation on a Cortex-M4 and provides 100k traces. Nkotto[14] subsequently mounted both template and CPA attacks on that dataset and recovered the decapsulation subkey, which illustrates the value of publishing artifacts of this kind.

Our dataset is complementary rather than redundant, and differs in three respects:

- **Measurement modality.** We provide near-field EM measurements, whereas that dataset provides power measurements. The two modalities differ in their noise and leakage characteristics, so publishing both allows researchers to check whether an analysis technique transfers from one to the other.
- **Optimised and protected code.** Their campaign covers a reference implementation only. We additionally cover `pqm4`, whose multiplication is hand-written Cortex-M4 assembly and is therefore closer to deployed code, and `mkm4`, which is first-order masked.
- **Per-share traces.** For `mkm4` we release the traces of both shares together with the corresponding masked key shares. This supports first-order leakage assessment, share-wise SNR analysis and the evaluation of multivariate or higher-order attacks, none of which can be studied on an unprotected implementation.

The dataset is also distributed as NumPy arrays rather than MATLAB files, which avoids a dependency on a proprietary toolchain.

3 Measurement Setup

3.1 Target Device and Firmware

We used the STM32F407G-DISC1[18] development board, with the STM32F407VG microcontroller running at 84MHz. Firmwares were built using the Arm GNU Toolchain 14.2.Rel1 with the following flags:

```
-mcpu=cortex-m4 -mthumb -mfloat-abi=hard -mfpv4-sp-d16
-00 -std=c99
```

All three implementations were built at -00 with the flags above. We report them so that the campaigns can be reproduced exactly.

3.2 Acquisition Chain

We used the Langer RF-K 7-4 near-field probe[4] and a Tektronix MSO54 oscilloscope. A GPIO trigger was used to start the acquisition. The oscilloscope was configured with a sampling rate of 6.25 GS/s, which corresponds to roughly 74 samples per clock cycle at the 84MHz core frequency. Acquisitions are single-shot: no averaging was applied.

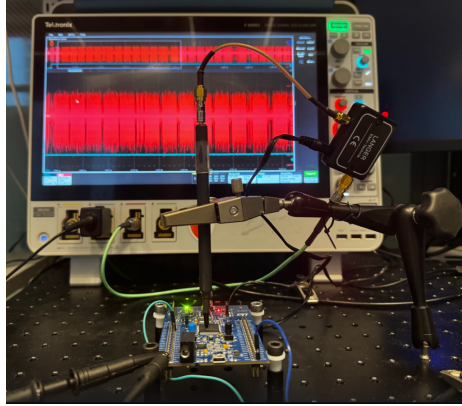


Fig.1: Measurement bench

3.3 Captured Window

Each trace is windowed on a *single* pair-pointwise multiplication and does *not* cover a full decapsulation. A GPIO pin is raised immediately before the first `basemul` call of `poly_basemul` and lowered immediately after it returns, and the oscilloscope is triggered on that edge. The captured window therefore corresponds to the multiplication of the coefficients a_0, a_1 of the key polynomial with

the coefficients b_0, b_1 of the ciphertext polynomial, that is, the first iteration of the loop shown in Listing 1.1. At 6.25 GS/s the 10k samples stored per trace span 1.6 μ s, or about 134 clock cycles at 84MHz, which comfortably brackets that multiplication. In `pqm4` and `mkm4` the same computation is performed by a hand-written assembly routine (`poly_frombytes_mul` and `basemul_asm` respectively), and the trigger brackets that routine instead.

For the masked `mkm4` implementation, the window is captured once per share, giving two 10k-sample traces per decapsulation, each the exact counterpart of the single window captured for the unprotected implementations. This is the origin of the two 10k-sample windows per trace reported in Table 1.

Three consequences are worth making explicit, as they determine how the traces should be interpreted:

- The traces contain only the targeted multiplication, which is why a comparatively small number of samples per trace is sufficient. Covering an entire decapsulation at the same sampling rate would instead require several orders of magnitude more samples per trace.
- Every trace has the same length, because the captured operation is a fixed-length instruction sequence independent of the ciphertext and of the key. In particular, the rejection sampling performed during the re-encryption step of decapsulation lies outside the captured window and therefore introduces neither length variation nor misalignment.
- Although the full 256 coefficients of both polynomials are provided as meta-data (Section 4.1), only a_0, a_1, b_0, b_1 are exercised inside the captured window.

The dataset stores the raw captures as returned by the oscilloscope: no decimation, resampling or time-domain compression is applied to the traces.

4 The Dataset

4.1 Structure and Format

Apart from the documentation, all files in the dataset are `.numpy` arrays. For each implementation there are two capture-scenario directories: `variable/` for variable decapsulation keys (for SNR computation) and `fixed/` for a fixed decapsulation key (for attacks). Each scenario contains 100k decapsulations. Within a capture-scenario directory, the `reference` and `pqm4` implementations follow the directory layout in Figure 2 (filenames are slightly different), while the masked `mkm4` implementation additionally splits the `b` polynomial and the trace into their two shares.

The captures are stored in chunks of 10000 consecutive decapsulations, so that each scenario consists of ten chunks. The index `i` in a file name is the chunk index, and *every* array of a given chunk has 10000 rows, one per decapsulation: row `j` of `traces_i.npy` is the trace of the same decapsulation as row `j` of `ap_i.npy`, `bp_i.npy`, `ct_i.npy` and, in the `variable` scenario, `dk_i.npy`.

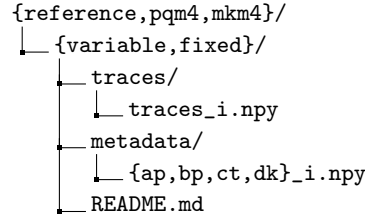


Fig. 2: Capture-scenario layout, common to the three implementations.

Traces and metadata are therefore aligned row by row, which is what makes the dataset directly usable for both profiled and unprofiled analysis.

The arrays of a chunk are interpreted as follows, where $s\{0,1\}$ indexes the share in the case of the masked `mkm4` implementation and the two figures given for `ct` and `dk` correspond to the 512 and 768 security levels respectively:

- `traces(_s{0,1})_i.npy`: (10000,10000) `int16`, the EM traces. For `mkm4` there is one such array per share.
- `ap_i.npy`: (10000,256) `int16`, the coefficients of the `a` polynomial.
- `bp_i.npy` / `bp_s{0,1}_i.npy`: (10000,256) `int16`, the coefficients of the `b` polynomial; for `mkm4` it is split per share `s`.
- `ct_i.npy`: (10000, {768,1088}) `uint8`, the ciphertexts.
- `dk_i.npy`: (10000, {1632,2400}) `uint8`, the decapsulation keys. In the `fixed` scenario the single key is stored once instead.
- `README.md`: per-implementation README.

Which of the two operands carries the secret differs between implementations: in the reference and `pqm4` implementations the decapsulation key is `a`, whereas in `mkm4` it is `b`, which is why `b` is the polynomial that is split into shares.

4.2 Dataset Contents

Table 1: Dataset statistics.

	Reference	<code>pqm4</code>	<code>mkm4</code> (masked)
Traces per scenario	100k	100k	100k
Traces in total	200k	200k	200k
Samples / trace	10k	10k	2×10k (one window per share)

4.3 Attacks Demonstrated on this Dataset

The traces released here are the ones we used in a series of practical key-recovery attacks, one per implementation, which we report in detail elsewhere. They establish that the dataset carries exploitable leakage for each of its three targets. In every case the attack targets the coefficients exercised inside the captured window (Section 3.3):

- On `pqm4`, a textbook correlation power analysis recovers the targeted secret coefficients from roughly 40 EM traces[12].
- On the reference implementation, a non-profiled deep-learning attack recovers secret coefficients without a clone device and without an explicit leakage model[10].
- On the masked `mkm4` implementation, first-order analysis fails as intended, but a second-order CPA on the centred cross-product of the two share leakages brings the correct coefficient to rank 0 after about 200 traces[11].

The masked case in particular is what the per-share traces are for, and it is the part of the dataset that cannot be reproduced from any of the existing public ML-KEM trace sets. Figure 3 shows why the second-order attack is possible: masking removes the secret from any single sample, but the leakage does not disappear, it is split between the two shares, and each share exhibits a clear signal-to-noise peak at a distinct point in time.

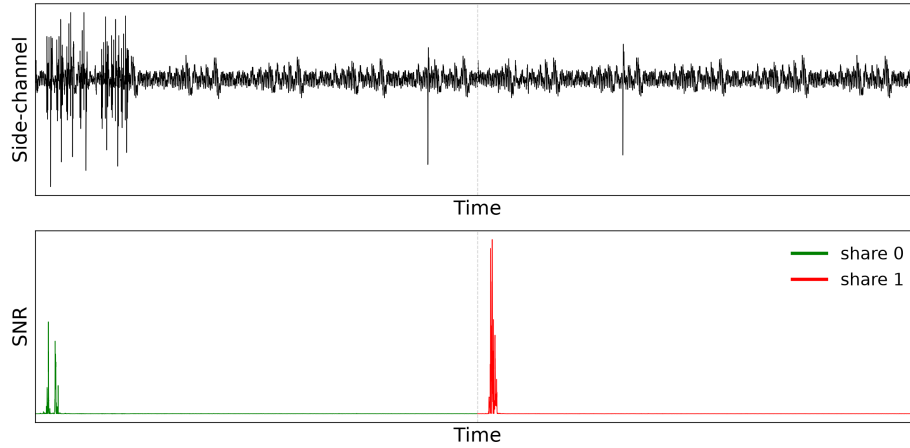


Fig. 3: Example EM trace (top) and the corresponding signal-to-noise ratio (bottom) on each share of the masked implementation. First share operates on the mask M and the second share on $sk - M$.

5 Availability

The dataset is available at the following URL:

https://pub-3483b4c265914de3a2a27c0a3b2076ee.r2.dev/d0nj0n_mlkem_dataset.zip. It is distributed under the GNU Lesser General Public License v3.0, in line with the other open-source tools released by our group. The archive contains a per-implementation `README.md` documenting the file layout and the semantics of each metadata array. The compressed archive is 12.6 GB.

As the URL above is not a persistent identifier, we welcome mirroring by the community.

6 Conclusion, Limitations and Future Work

The dataset presented in this work enables side-channel research on some of the most used implementations of the ML-KEM algorithm, including a masked one. We hope it will help in the security assessment of the various implementations of this algorithm.

6.1 Limitations

The current release is deliberately narrow in scope, and we state its boundaries explicitly so that users know what the traces can and cannot support. It covers a single operation, the pair-pointwise polynomial multiplication, on a single device and at a single probe position. The restriction to decapsulation is deliberate: it is the operation in which the long-term decapsulation key is an operand, whereas encapsulation involves only the public key and ephemeral randomness. Only one security level is available per implementation, and the levels do not match across implementations because `mkm4` exists only for the 768 parameter set. All firmwares are built at -00. Finally, the ciphertexts used during the campaigns are honestly generated: the dataset therefore does not support attacks that require specifically crafted ciphertexts.

6.2 Future Work

We see the following extensions as the most valuable, roughly in decreasing order of expected usefulness to the community:

- **More recent masked implementations.** In particular the bitsliced masked implementations of `pqm4_masked`[17], which represent the current state of the art in masked ML-KEM on Cortex-M4, and higher-order masking.
- **Other target operations within decapsulation.** Most importantly the NTT and the masked ciphertext comparison, both of which are established targets and exercise very different leakage behaviour from the multiplication.

- **Crafted ciphertexts.** Releasing campaigns with chosen valid and invalid ciphertexts would support plaintext-checking and decapsulation-failure oracle attacks[15], which are among the most efficient known key-recovery strategies against ML-KEM.
- **Build and noise variants.** The same campaigns compiled at `-O2` and `-Os` to quantify the effect of compiler optimisation on exploitability, and captures at different noise levels.
- **Additional and matching security levels,** so that the unprotected and masked implementations can be compared at an identical parameter set.

References

1. Aysu, A., Dinu, D., Gaj, K., Ganji, F., Hashemi, M., J, R., Mehta, D., Saarinen, M.J.O., Schaumont, P., Tol, C.: Open tools, interfaces and metrics for implementation security testing: Testing post-quantum cryptography implementation security. Online (2025), <https://optimist-ose.org/docs/pqc/intro>, working Document, version 0.5
2. Bos, J., Ducas, L., Kiltz, E., Lepoint, T., Lyubashevsky, V., Schanck, J.M., Schwabe, P., Seiler, G., Stehlé, D.: CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM. In: 2018 IEEE European Symposium on Security and Privacy (EuroS&P). pp. 353–367 (2018). <https://doi.org/10.1109/EuroSP.2018.00032>
3. CRYSTALS Team: Kyber: Reference implementation. <https://github.com/pq-crystals/kyber> (2024), accessed: 2026-08-26
4. EMV, L.: Rf-k 7-4 em probe. <https://www.langer-emv.com/en/download/35/9/rf-k-7-4-h-field-probe-30-mhz-up-to-1-ghz.pdf> (2026)
5. Ghiban, C., Choudary, M.O.: Improved correlation power analysis attack on the latest cortex m4 kyber implementation. *Cryptography* **9**(1) (2025). <https://doi.org/10.3390/cryptography9010019>, <https://www.mdpi.com/2410-387X/9/1/19>
6. Heinz, D., Kannwischer, M.J., Land, G., Pöppelmann, T., Schwabe, P., Sprenkels, D.: First-order masked Kyber on ARM Cortex-M4. *IACR Cryptology ePrint Archive Report* **2022/058** (2022), <https://eprint.iacr.org/2022/058>
7. Ji, Y., Dubrova, E.: A side-channel attack on a masked hardware implementation of CRYSTALS-kyber. *Cryptology ePrint Archive, Paper 2023/1084* (2023). <https://doi.org/10.1145/3605769.3623992>, <https://eprint.iacr.org/2023/1084>
8. Kannwischer, M.J., Rijneveld, J., Schwabe, P., Stoffelen, K.: pqm4: Testing and benchmarking NIST PQC on ARM Cortex-M4. <https://github.com/mupq/pqm4> (2019), second NIST PQC Standardization Conference. Accessed: 2026-08-26
9. Karlov, A., de Guertechin, N.L.: Power analysis attack on kyber. *Cryptology ePrint Archive, Paper 2021/1311* (2021), <https://eprint.iacr.org/2021/1311>
10. Ledger Donjon: Non-profiled deep learning side-channel attacks on ML-KEM. <https://donjon.ledger.com/blog/non-profiled-deep-learning-sca-ml-kem/> (2026), accessed: 2026-08-27
11. Ledger Donjon: A second-order side-channel attack on masked Kyber768. <https://donjon.ledger.com/blog/second-order-side-channel-attack-masked-kyber768/> (2026), accessed: 2026-08-27
12. Ledger Donjon: Side-channel attacks and post-quantum cryptography. <https://www.ledger.com/blog/risk-side-channel-attacks-post-quantum-cryptography> (2026), accessed: 2026-08-27

13. National Institute of Standards and Technology: Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication (FIPS) 203 (2024). <https://doi.org/10.6028/NIST.FIPS.203>, <https://doi.org/10.6028/NIST.FIPS.203>
14. Nkotto, S.: Template and CPA side channel attacks on the Kyber/ML-KEM pair-pointwise multiplication. Cryptology ePrint Archive, Paper 2025/1577 (2025), <https://eprint.iacr.org/2025/1577>
15. Ravi, P., Roy, S.S., Chattopadhyay, A., Bhasin, S.: Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs. IACR Transactions on Cryptographic Hardware and Embedded Systems **2020**(3), 307–335 (2020). <https://doi.org/10.13154/tches.v2020.i3.307-335>
16. Rezaeezade, A., Yap, T., Jap, D., Bhasin, S., Picek, S.: Side-channel power trace dataset for Kyber pair-pointwise multiplication on Cortex-M4. Cryptology ePrint Archive, Paper 2025/811 (2025), <https://eprint.iacr.org/2025/811>
17. SIMPLE-Crypto Association: `pqm4_masked`: Masked ML-KEM implementations for ARM Cortex-M4. https://github.com/simple-crypto/pqm4_masked (2024), accessed: 2026-08-27
18. STMicroelectronics: `Stm32f407g-disc1`. https://www.st.com/resource/en/user_manual/um1472-discovery-kit-with-stm32f407vg-mcu-stmicroelectronics.pdf (2026)